APC 机制与 NtQueueApcThread 注入原理

上章我们理解了模块踩踏的蓝队检测视角，发现最大暴露点是 CreateRemoteThread。本章开始进入 APC 注入 + Early Bird。

核心思路：不建新线程，而是把 Shellcode 挂到目标进程已有线程的 APC 队列里，等它进入"可提醒等待状态"时自动执行。

- **什么是 APC**

APC = Asynchronous Procedure Call（异步过程调用）。

你可以把它理解成"线程的待办事项列表"。每个线程都有一张 APC 队列，系统会在特定时机检查这张队列，发现有待办事项就执行。

- **线程的两种状态**

状态	物理意义	系统是否检查 APC 队列
可提醒状态（Alertable）	线程主动说："我现在没事干，有什么待办事项可以执行。"	检查并执行
不可提醒状态	线程正在忙，或者在系统调用中不可被打断。	不检查

关键 API 触发的"可提醒状态"：

- ✓ SleepEx(dwMilliseconds, TRUE)：第二个参数 TRUE 表示"可提醒"。
- ✓ WaitForSingleObjectEx(hHandle, dwMilliseconds, TRUE)：第三个参数 TRUE 表示"可提醒"。
- ✓ WaitForMultipleObjectsEx(..., TRUE)：同理。

注：普通 Sleep() 和 WaitForSingleObject()进入的是不可提醒状态，APC 不会被执行。

- **APC 队列的物理位置**

每个线程有个 KAPC_STATE 结构，里面维护了两个 APC 队列：

- ✓ 内核态 APC 队列：由内核插入（比如 I/O 完成通知）。
- ✓ 用户态 APC 队列：由用户态通过 QueueUserAPC 或 NtQueueApcThread 插入。

我们关心的是用户态 APC 队列。

- **APC 执行的时机**

```
线程调用 SleepEx(1000, TRUE)
    ↓
线程进入可提醒等待状态
    ↓
系统检查用户态 APC 队列
    ↓
发现有 APC 待执行
    ↓
切换到用户态，执行 APC 回调函数
    ↓
回调函数返回后，继续等待或返回
```

- **APC 注入的核心流程**

- 在目标进程申请内存，写入 Shellcode（Module Stomping 或 VirtualAllocEx）
- 找到目标进程里的一个线程 ID
- 用 OpenThread 打开这个线程
- 用 NtQueueApcThread 把 Shellcode 地址插入该线程的 APC 队列
- 等待目标线程进入可提醒状态（比如它调用了 SleepEx）
- Shellcode 被执行

关键优势：

- 不创建新线程 → 避开 EDR 的"远程线程创建"监控。
- 执行发生在已有线程里 → 调用栈看起来更自然。

关键劣势：

- 依赖目标线程主动进入可提醒状态。如果目标线程从不调用 SleepEx / WaitForSingleObjectEx，APC 永远不会执行。

解决方案：用 Early Bird 技术——在进程启动初期就注入 APC，那时候线程还没开始干活，必然有机会进入可提醒状态。

- **NtQueueApcThread 详解**

函数原型

```
NTSTATUS NtQueueApcThread(
    HANDLE ThreadHandle,              // 目标线程句柄
    PKNORMAL_ROUTINE ApcRoutine,   // APC 回调函数地址
    PVOID ApcArgument1,               // 回调函数参数 1
    PVOID ApcArgument2,               // 回调函数参数 2
    PVOID ApcArgument3                 // 回调函数参数 3
);
```

参数含义

参数	物理意义
ThreadHandle	目标线程的句柄（通过 OpenThread 获得）
ApcRoutine	要执行的函数地址（我们填 Shellcode 的地址）
ApcArgument1	传递给 APC 回调的第一个参数（我们传 NULL）
ApcArgument2	第二个参数（传 NULL）
ApcArgument3	第三个参数（传 NULL）

注：ApcRoutine 的函数签名必须是 void CALLBACK AapcRoutine(ULONG_PTR param)。但如果我们只传 NULL，Shellcode 就不需要关心参数。

- **为什么用 NtQueueApcThread 而不是 QueueUserAPC**
  - ✓ QueueUserAPC：是 kernel32.dll 导出的 API，容易被 EDR Hook。
  - ✓ NtQueueApcThread：是 ntdll.dll 导出的 Native API，更底层，Hook 难度更高。
- **代码示例（见文件夹）**
- **第一版：明文 API 实现**
- **第二版：无 IAT 版本（用封装的 .h 文件）**
- **如何让 APC 真正执行**

写完代码后，可能会发现：注入成功了，但 Shellcode 没执行。这是因为目标线程还没进入“可提醒状态”。

测试方法：写一个简单的测试目标程序：

```
// target.cpp
include <windows.h>
include <iostream>

int main() {
    std::cout << "Target PID: " << GetCurrentProcessId() << std::endl;
    while (true) {
        SleepEx(5000, TRUE);   // 关键：进入可提醒状态
        std::cout << "醒来一次" << std::endl;
    }
}
```

编译运行后，用 APC 注入器对它注入。目标线程会在 SleepEx 时执行 APC，Shellcode 就被执行了。

观察方式：

如果 Shellcode 是 mov eax, 42; ret，你看不到任何效果，但它确实执行了。

改成弹窗 Shellcode，就能看到效果。